

第 6 次课教师讲义：函数调用约定与栈帧

技术编号：week06 | 建议检查点：week06-stack-abi
系统化过程调用，彻底读懂第 1-2 次课的 bl / jr \$ra。

0. 一句话目标

掌握 LoongArch 教学用调用约定与最小栈帧，能解释叶子/非叶子函数，并能对照 clear_bss、kernel_main 与 exception_entry。

1. 课程定位 (Why)

第 1-2 次课反复出现：

```
bl    clear_bss
bl    kernel_main
jr    $ra
```

第 5 次课解决了函数内部的循环结构；本次课解决函数之间如何交接：

参数在哪？返回值在哪？谁保存 \$ra？栈帧长什么样？

2. 教学目标

- 1. 记住：整型/指针参数优先走 \$a0-\$a7，返回值主要在 \$a0。
- 2. 理解 bl 写入 \$ra，jr \$ra 返回。
- 3. 区分叶子函数与非叶子函数；非叶子必须保存 \$ra。
- 4. 会画最小栈帧：伸栈 → 存 \$ra → 体 → 取 \$ra → 折栈 → 返回。
- 5. 能解释 exception_entry 中保存 \$ra 的原因（为第 12 次课铺垫）。

3. 课前准备

- 复习第 3 次课寄存器角色、第 4 次课 \$sp+offset 访存。
- 打开 lib/string.S 的 memset 入口与 boot/start.S 的 exception_entry。

4. 课程知识

4.1 调用约定 (LP64 教学简化版)

内容	位置
第 1-8 个整数/指针参数	\$a0-\$a7
返回值	\$a0 (教学先掌握这一个)
返回地址	\$ra (由 bl 设置)
栈	\$sp，向低地址增长；注意对齐习惯（如 16 字节）

caller-saved / callee-saved (教学版)

- \$t*：调用者不指望跨调用存活
- \$s* / 部分帧指针：被调用者若使用需保存（本课点到，实验可先少用）

4.2 叶子函数

```
# int add_one(int x) → $a0 = x, 返回值 $a0
add_one:
    addi.d    $a0, $a0, 1
    jr        $ra
```

汇编精讲

1. 不调用其他函数, `$ra` 不会被覆盖。
2. 参数已在 `$a0`, 结果写回 `$a0`。
3. `jr $ra`: 跳到返回地址。

4.3 非叶子函数

```
foo:
    addi.d    $sp, $sp, -16
    st.d      $ra, $sp, 8
    bl        bar                # 这里会改写 $ra
    ld.d      $ra, $sp, 8
    addi.d    $sp, $sp, 16
    jr        $ra
```

汇编精讲

1. `addi.d $sp, $sp, -16`: 向低地址分配栈帧。
2. `st.d $ra, $sp, 8`: 保存返回地址。
3. `bl bar`: 进入子函数; 返回后应回到 `ld.d` 处。
4. 对称恢复 `$ra` 与 `$sp`, 再 `jr $ra`。

关键结论: 若漏存 `$ra`, 嵌套调用后无法返回正确位置。

4.4 对照工程代码

```
bl    clear_bss
bl    kernel_main
```

`_start` 中的两次调用

- `clear_bss` 必须以 `jr $ra` 正确返回, 第二次 `bl` 才有意义。
- `_start` 本身在 `kernel_main` 返回后进入 `halt`, 不依赖再返回“更上层”。

```
memset:                                # void *memset(void *s, int c, size_t n)
    move     $t0, $a0                  # 保存 dst 以便返回
    ...
    move     $a0, $t0
    jr       $ra
```

`memset` 的参数槽

C 参数	寄存器
s	\$a0
c	\$a1
n	\$a2
返回 s	\$a0

```

addi.d $sp, $sp, -32
st.d   $ra, $sp, 24
...
bl     exception_handler
ld.d   $ra, $sp, 24
...
ertn                                # 注意：异常返回不是 jr $ra

```

exception_entry 预告 模式相同：为调用 C 保存 \$ra；最终返回路径不同（ertn）。

5. 课堂 Demo

1. 叶子 vs 非叶子：板书 \$ra 时间线。
2. 让学生在 memset 上标注三个参数寄存器。
3. (可选) GDB 在 bl 前后打印 \$ra。

6. 实验实践

Task A

实现 int add3(int a, int b, int c) 汇编版, C 中调用并 printk 结果 (或返回后用已知方式观察)。

Task B

给 foo→bar 嵌套画栈帧草图。

Task C

判断：下列函数是否必须保存 \$ra？

- 只做 addi 后返回
- 内部 bl printk

7. AI 共学

允许检查栈帧是否成对；要求学生能讲解“为何非叶子要存 \$ra”。

8. 思考与拓展

1. 栈溢出典型症状有哪些？
2. 变参函数为何更麻烦（点到为止）？
3. 内联函数如何减少 bl/jr？

9. 板书

参数 \$a0..\$a7 返回 \$a0 返回地址 \$ra
bl = 调走并记下回家路
非叶子: sp↓ 存 ra bl 取 ra sp↑ jr ra
回看: clear_bss / memset / exception_entry